

实验 10 FIFO 页面置换算法

1. 实验目的

通过请求页式存储管理中的先进先出（FIFO）算法的模拟设计，理解 FIFO 算法的设计思想及 FIFO 算法的实现方法。

2. 相关知识

(1) FIFO 算法的基本思想

在请求页式管理中，当发生缺页中断且主存没有空闲页面时，总是淘汰最先进入主存的页面，即选择在主存中驻留时间最久的页面被淘汰。

(2) FIFO 算法实现原理

把一个进程已调入主存的页面按先后次序链接成一个队列，并设置一个指针即可。该算法是一种最直观但性能较差的页面置换算法，并可能会有 BELADY 异常现象，即分配的页面数增加时，缺页中断次数反而增加。

3. 实验要求

假如某进程 P 有 5 个页，进程访问页的顺序为：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5；如果在内存中分配给该进程 3 个页面，给出缺页中断次数和页面的淘汰顺序，示例如下：

页面	1	2	3	4	1	2	5	1	2	3	4	5
页面 1	1	2	3	4	1	2	5	5	5	3	4	4
页面 2		1	2	3	4	1	2	2	2	5	3	3
页面 3			1	2	3	4	1	1	1	2	5	5
缺页	Y	Y	Y	Y	Y	Y	Y	N	N	Y	Y	N

4. 实验程序

【程序 4_1】FIFO 算法实验程序。

```
1. #include <stdio.h>
2. void main(){
3.     int i,j,k = 0;
4.     char cc[13];          //Y 表示产生缺页中断，N 表示未产生缺页中断
5.     int a[12] = {1,2,3,4,1,2,5,1,2,3,4,5};    //页面调度顺序
6.     int b[3][13],c[13],p = 0;
7.     b[0][0] = 0;          //C 语言定义数组，其初值是不确定的
8.     b[1][0] = 0;
9.     b[2][0] = 0;
10.    printf("      ");
11.    for(i = 0;i < 12;i++){
12.        printf("%6d",a[i]);
13.        printf("\n      =====
=====\\n");
14.        for(i = 0;i < 12;i++){
15.            if(a[i]==b[0][i]||a[i]==b[1][i]||a[i]==b[2][i]){
16.                b[0][i+1]=b[0][i];
17.                b[1][i+1]=b[1][i];
```

```

18.         b[2][i+1]=b[2][i];
19.         cc[i]='N';
20.     }
21.     else{
22.         if(i > 2){
23.             c[p]=b[2][i];
24.             p++;
25.         }
26.         b[0][i+1]=a[i];
27.         b[1][i+1]=b[0][i];
28.         b[2][i+1]=b[1][i];
29.         cc[i]='Y';
30.         k=k+1;
31.     }
32. }
33. for(j=0;j<13;j++)
34.     printf("%6d",b[0][j]);
35. printf("\n      -----
-----\n");
36. for(j=0;j<13;j++)
37.     printf("%6d",b[1][j]);
38. printf("\n      -----
-----\n");
39. for(j=0;j<13;j++)
40.     printf("%6d",b[2][j]);
41. printf("\n      -----
-----\n");
42. printf("      ");
43. for(j=0;j<13;j++){
44.     printf("      ");
45.     putchar(cc[j]);
46. }
47. printf("\n 缺页中断次数: %3d\n",k);
48. printf("页面淘汰顺序: ");
49. for(j=0;j<p;j++)
50.     printf("%3d",c[j]);
51. printf("\n\n");
52. }

```

5. 【程序运行结果截图】

	1	2	3	4	1	2	5	1	2	3	4	5
=====												
0	1	2	3	4	1	2	5	5	5	3	4	4

0	0	1	2	3	4	1	2	2	2	5	3	3

0	0	0	1	2	3	4	1	1	1	2	5	5

	Y	Y	Y	Y	Y	Y	Y	N	N	Y	Y	N
缺页中断次数:	9											
页面淘汰顺序:	1 2 3 4 1 2											

6. 实验思考与扩充

将【程序 4_1】改为 4 个页面，运行后的结果与【程序 4_1】的运行结果进行对比。

实验 11 LRU 页面置换算法

1. 实验目的

通过请求页式存储管理中的最近最久未使用页面置换算法（LRU）的模拟设计，进一步理解 LRU 算法的基本思想，给出 LRU 算法的实现原理和方法，并具体编程实现该算法的模拟程序。

2. LRU 算法的基本思想

该算法的基本思想是：当需要淘汰某一页时，选择离当前时间最近的一段时间内最久没有使用过的页被淘汰。该算法的出发点是，如果某页被访问了，则它可能马上还要被访问；或者反过来说，如果某页很长时间未被访问，则它在最近一段时间也不会被访问。

3. LRU 算法实现原理

当某进程发生缺页中断且主存没有空闲页面时，为了选择离当前时间最近的一段时间内最久没有被访问的页被淘汰，可设计一种数据结构 $c[5][2]$ ，第一列表示页号，共有 5 页（页号 1~5），第二列表示每个页的计数值，其初值均为 0。

当访问某页时，如果该页在内存，则取计数值最大的且加 1，作为该页的计数值；如果该页不在内存且内存有空闲页面，则将该页调入内存，且取计数值最大的且加 1，作为该页的计数值；如果该页不在内存且内存没有空闲页面时，淘汰计数值最小的页（计数值最大的是刚刚被访问的页面，计数值最小的就是最近最久未被访问的页面），并将被淘汰页的计数值清零，然后将该页调入内存，并取计数值最大的且加 1，作为该页的计数值。

4. 实验要求

假如某进程 P 有 5 个页，进程访问页的顺序为：4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5；如果在内存中分配给该进程 3 个页面，示例如下：

页面	4	3	2	1	4	3	5	4	3	2	1	5
页面 1	4	3	2	1	4	3	5	4	3	2	1	5
页面 2		4	3	2	1	4	3	5	4	3	2	1
页面 3			4	3	2	1	4	3	5	4	3	2
缺页	Y	Y	Y	Y	Y	Y	Y	N	N	Y	Y	Y

5. 实验程序

【程序 4_2】LRU 算法实验程序。

```
53. #include <stdio.h>
54.
55. void max_value(int x,int cc[][2]); //函数声明，页表处理
56. int r_algorithm(int cc[][2]); //函数声明，选择页面淘汰算法
57. char cc[13]; //Y 表示产生缺页中断，N 表示未产生缺页中断
58. void page_table(int page1,int c[5][2]); //打印页表
59.
60. void main(){
61. int i,j,page,row = 0,col = 1; //b[row][col],行/列指针
62. int k = 0; //记录缺页中断次数
```

```

63. int a[12] = {4,3,2,1,4,3,5,4,3,2,1,5}; //存放页的调度顺序
64. int b[3][13]; //模拟内存（分配三个页面）
65. int c[5][2] = {{1,0},{2,0},{3,0},{4,0},{5,0}}; //定义页表并赋初值
66. int d[13],p = 0; //存放页面淘汰顺序，p 页面淘汰数组 d 的指针
67. b[0][0] = 0;
68. b[1][0] = 0;
69. b[2][0] = 0;
70.
71. //*****页面调度处理*****
72. for(i = 0;i < 12;i++){
73.     if(a[i]==b[0][i]||a[i]==b[1][i]||a[i]==b[2][i]){
74.         b[0][i+1] = b[0][i]; //将前一列数据复制到下一列
75.         b[1][i+1] = b[1][i];
76.         b[2][i+1] = b[2][i];
77.         max_value(a[i],c); //处理页表，a[i]页面是刚被访问的页面
78.         page_table(a[i],c); //打印页表
79.         cc[i] = 'F';
80.         col++; //col 指向下一列
81.     }
82.     else{ //页面不在内存
83.         if(row > 2){ //row > 2 表示内存已没有空闲页面
84.             page = r_algorithm(c); //返回淘汰的页面 page
85.             d[p] = page; //d[]存放被淘汰的页面
86.             p++;
87.             k++; //缺页中断次数
88.             b[0][i+1] = b[0][i]; //将前一列数据复制到下一列
89.             b[1][i+1] = b[1][i];
90.             b[2][i+1] = b[2][i];
91.             cc[i] = 'Y';
92.             if(b[0][i+1] == page)
93.                 b[0][i+1] = a[i];
94.             if(b[1][i+1] == page)
95.                 b[1][i+1] = a[i];
96.             if(b[2][i+1] == page)
97.                 b[2][i+1] = a[i];
98.             max_value(a[i],c); //访问 a[i]页面，i 页面是刚被访问的页面
99.             page_table(a[i],c); //打印页表
100.        }
101.        else{
102.            b[0][i+1] = b[0][i]; //将前一列数据复制到下一列
103.            b[1][i+1] = b[1][i];
104.            b[2][i+1] = b[2][i];
105.            cc[i] = 'Y';
106.            b[row][col] = a[i]; //a[i]页面进入内存

```

```

107.          max_value(a[i],c);          //访问 a[i]页面, i 页面是刚被访问的页
        面
108.          col++;
109.          k++;          //缺页中断次数
110.          row++;
111.          page_table(a[i],c);          //打印页表
112.      }
113.  }
114.  }
115.  //=====显示处理结果=====
116.  printf("\n      ");
117.  for(i = 0;i < 12;i++)
118.      printf("%6d",a[i]);          //显示页面调度顺序
119.  printf("\n
=====\\n"
    );
120.  for(j = 0;j < 13;j++)
121.      printf("%6d",b[0][j]);
122.  printf("\n      -----
-----\\n");
123.  for(j = 0;j < 13;j++)
124.      printf("%6d",b[1][j]);
125.  printf("\n      -----
-----\\n");
126.  for(j = 0;j < 13;j++)
127.      printf("%6d",b[2][j]);
128.  printf("\n      -----
-----\\n");
129.  printf("      ");
130.  for(j = 0;j < 13;j++){          //打印是否产生缺页中断
131.      printf("      ");
132.      putchar(cc[j]);
133.  }
134.  printf("\n 缺页中断次数: %4d\\n",k);
135.  printf("\n 页面淘汰顺序: ");
136.  for(j = 0; j < p;j++)
137.      printf("%3d",d[j]);          //显示页面淘汰顺序
138.  printf("\n\\n");
139. }
140.
141. //=====访问的页面在内存的处理（页表处理）=====
142. void max_value(int x,int cc[][2]){//x-页号: 求页表中计数的最大值, 并将该页面置
    为最新访问页面
143.     int i, max;

```

```

144.     max = cc[0][1];
145.     for(i = 0;i < 5;i++)
146.         if(max < cc[i][1])
147.             max = cc[i][1];
148.     for(i = 0; i < 5;i++)
149.         if(cc[i][0] == x)
150.             cc[i][1] = max + 1;
151. }
152.
153. //===== 选择被淘汰的页面（页表处理）=====
154. int r_algorithm(int cc[5][2]){
155.     int i,min,row,p;
156.     for(i = 0;i < 5;i++)    //查询第一个计数为非0的页面的计数值
157.         if(cc[i][1] != 0){
158.             min = cc[i][1];
159.             p = cc[i][0];
160.             row = i;
161.             break;
162.         }
163.     for(i = 0;i < 5;i++) //寻找计数值最小的页面
164.         if(min > cc[i][1]&&cc[i][1] != 0){
165.             min = cc[i][1];
166.             p = cc[i][0];    //最小数所对应的页号被淘汰
167.             row = i;        //记录最小数所在的行
168.         }
169.     cc[row][1] = 0;        //在页表中被淘汰的页面计数清零
170.     return(p);            //返回被淘汰的页面-P
171. }
172.
173. // =====打印页表=====
174. void page_table(int page1,int c[5][2]){
175.     int i;
176.     printf("页面调度顺序 page = %d\n",page1);
177.     for(i = 0; i < 5;i++)
178.         printf("%5d%5d\n",c[i][0],c[i][1]);
179. }

```

6. 【程序运行结果截图】

	4	3	2	1	4	3	5	4	3	2	1	5
0	4	4	4	1	1	1	5	5	5	2	2	2
0	0	3	3	3	4	4	4	4	4	4	1	1
0	0	0	2	2	2	3	3	3	3	3	3	5
	Y	Y	Y	Y	Y	Y	Y	F	F	Y	Y	Y
缺页中断次数：	10											
页面淘汰顺序：	4 3 2 1 5 4 3											

7. 实验思考与扩充

- (1) 修改【程序 4_2】，将分配给该进程 3 个页面改为 4 个页面，并对运行结果进行分析与对比。
- (2) 如果 LRU 算法实现原理应用于 FIFO 算法，则 FIFO 页面置换算法的实验程序应该如何修改？
- (3) 如何实现 LRU 两个近似算法：最不经常使用的页面淘汰算法（LFU）和最近没使用的页面淘汰算法（NUR）？